Project 2 - xv6 OS Assignment

Group members: Ankrit Gupta (solo submission)

Date: October 21, 2026

Same group as Project 1? Yes. I worked alone on both projects.

Environment

- **OS:** Windows Subsystem for Linux 2 (WSL2), Ubuntu, on Windows 11
- **Kernel:** Linux DESKTOP-IGT6P4O 6.18.33.2-microsoft-standard-WSL2
- **Compiler:** gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
- **Emulator:** QEMU emulator version 10.2.1 (Debian 1:10.2.1+ds-1ubuntu3.2)
- **xv6 source:** github.com/mit-pdos/xv6-public (x86)
- **Special installation steps:** none (same setup as Project 1)
- **Build and run:** make qemu-nox [SCHEDULER=DEFAULT|SJF|PRIORITYRR] [QUANTUM=n] [CPUS=1]

How to build and run each scheduler

```
make qemu-nox SCHEDULER=DEFAULT
make qemu-nox SCHEDULER=SJF
make qemu-nox SCHEDULER=PRIORITYRR
make qemu-nox SCHEDULER=PRIORITYRR QUANTUM=10
```

- DEFAULT is the original xv6 round-robin scheduler (used if SCHEDULER is omitted). SJF is Shortest Job First; brackets such as SCHEDULER=[SJF] also work. PRIORITYRR is the multi-level priority scheduler with round robin; QUANTUM=n sets its time slice in timer ticks.
- I used CPUS=1 for all measurements so runs are repeatable (only CPU 0 effectively runs in my setup).
- **Makefile changes:** the SCHEDULER and QUANTUM variables are checked (an unknown scheduler name or a non-numeric QUANTUM stops the build with an error) and passed to the compiler as -D<SCHEDULER> and -DRR_QUANTUM=<QUANTUM>. A stamp file (.sched_<name>_q<quantum>, removed by make clean) makes proc.o and trap.o rebuild when the scheduler changes, so make clean is not needed between schedulers. Also in the Makefile: _sort, _tickstest, _sjftest, _timeit and _priotest were added to UPROGS, and os611.txt and nums.txt are copied onto the disk image (TESTFILES).
- **Other change:** FSSIZE in param.h was raised from 1000 to 2000 blocks. With all the new programs the original 1000-block disk was almost full, and stressfs then crashed with "balloc: out of blocks".
- **Files added:** sort.c, tickstest.c, sjftest.c, timeit.c, priotest.c, os611.txt, nums.txt
- **Files modified:** grep.c, Makefile, param.h, proc.h, proc.c, trap.c, defs.h, syscall.h, syscall.c, sysproc.c, user.h, usys.S, .gitignore

Part 1 – sort and grep

sort (sort.c): reads a file, or standard input when no file is given (so cat file | sort works), and prints the lines in ASCII order.

- -r: reverse order.
- -n: numeric. Compares the leading non-negative integer of each line using xv6's atoi, so negative numbers are not supported.
- -u: drops lines identical to the previous sorted line.

- Flags are given separately, for example sort -r -n file or sort -n -u file.
- Limits: 64 KB of input, 2048 lines, insertion sort (fine for small files). It prints nothing extra.

grep (grep.c): I extended xv6's existing grep (its Kernighan and Pike matcher for ^ . $ * is kept unchanged).

- Flags: -i (case-insensitive), -n (line numbers), -v (invert), and combined flags such as -in and -vn.
- Reads standard input when no file is given, and prints the file name before each match when more than one file is searched.
- Lines up to 1023 characters are handled. Usage: grep [-inv] pattern [file ...].

Test files: os611.txt and nums.txt are created on the host and copied into fs.img by the Makefile.

xv6 shell notes that affected testing: no >> append, no quote handling (write patterns without quotes), a 128-character console line limit, and file names are cut to 14 characters.

Part 2 – ticks_running(pid)

- struct proc has a new field run_ticks, set to 0 in allocproc().
- In trap.c, every timer interrupt adds one to run_ticks of the process that was RUNNING.
- procticks(pid) in proc.c (holding ptable.lock) returns run_ticks, or -1 if no process with that pid exists. A process that exists but has not run yet returns 0. A zombie that has not been waited for can still be queried.
- sys_ticks_running() (sysproc.c) reads the argument with argint() and calls procticks(). Syscall number 23; plumbing in syscall.h, syscall.c, user.h and usys.S.
- Test program: tickstest.c (the calling process, a child before and after it runs and after wait(), init and sh, and pids 9999, -1 and 0).

Part 3 – Shortest Job First (SCHEDULER=SJF)

- **Predicted job length:** each new process gets a pseudo-random length from 1 to 100 ticks in allocproc(), from a linear congruential generator, so lengths vary from process to process but repeat from boot to boot. All constants are named in param.h: SJF_MINLEN (1), SJF_MAXLEN (100), SJF_SEED (12345), SJF_LCG_A (1664525), SJF_LCG_C (1013904223).
- scheduler() (proc.c) picks the RUNNABLE process with the smallest job_len (ties go to the lowest slot in the process table).
- SJF is non-preemptive: trap.c does not call yield() on timer ticks, so a process keeps the CPU until it sleeps or exits.
- sjf_job_length(pid) (syscall 24) returns the predicted length, or -1 if the pid does not exist or is not RUNNABLE or RUNNING.
- Test programs: sjftest.c (prints each child's predicted length and the order and tick at which they finish) and timeit.c (runs a command and prints the ticks it took).

- **Result:** under DEFAULT the children of sjftest finish together; under SJF they finish one at a time, shortest predicted job first.

Part 4 – Multi-level priority with round robin (SCHEDULER=PRIORITYRR)

- **Levels:** PRIORITY_LEVELS = 3 (param.h), numbered 0 (highest) to 2 (lowest). New processes start at PRIORITY_DEFAULT = 1, and a child starts at its parent's priority (fork).
- scheduler() picks the highest non-empty level. Inside a level it is round robin: the search starts just after the table slot that ran last at that level (rr_last[]).
- **Quantum:** RR_QUANTUM (param.h), default 5 timer ticks. It can be changed at build time with QUANTUM=n; I tested 5 and 10. **Quantum chosen for the submission: 5 ticks.**
- **Preemption** (trap.c and sched_should_yield() in proc.c): on each timer tick the running process is preempted if its quantum is used up or if a higher-priority process is RUNNABLE.
- set_sched_priority(p) (syscall 25) sets the calling process's level; it returns 0, or -1 if p is outside 0..2. get_sched_priority(pid) (syscall 26) returns the level, or -1 if the pid does not exist or is not RUNNABLE or RUNNING.
- Under DEFAULT and SJF the two syscalls still work, but the priority is ignored.
- **Known limitation:** strict priority with no aging, so a process that never sleeps at a high priority starves lower levels.
- Test program: priotest.c checks the syscall return values, then runs six children (two at each level), each using 30 ticks of CPU. **Result:** level 0 finishes first, then 1, then 2; the two children inside a level finish at the same tick because they take turns. "slices" counts how often a child was taken off the CPU: about 5 with quantum 5 and 2 to 3 with quantum 10.

Scheduler comparison

Ticks reported by timeit, one run each, CPUS=1:

Command	DEFAULT	SJF	PRIORITYRR
timeit stressfs	47	44	38
timeit ls	41	41	39

Single runs vary a lot (stressfs ranged from about 30 to 80 ticks across repeated runs on the same scheduler), so these small differences are within noise. The commands uniq and find named in the assignment do not exist in this xv6, so I timed stressfs and ls and checked that pipes and fork still work.

- **stressfs, ls:** no meaningful difference. SJF and round robin only behave differently when several processes are runnable at once; single short commands barely differ. SJF does fewer forced context switches (no timer preemption), which may save a few ticks in stressfs.
- **Pipes and fork:** cat README | wc, cat os611.txt | grep -v I and forktest all ran correctly under all three schedulers, and so did sort -r -n, sort -u and grep -in on the test files. Pipe writers and readers sleep, so SJF's lack of preemption does not hurt them.

- **Effects on existing functionality:** SJF is non-preemptive, so a CPU-bound process that never sleeps would block everything else (including the shell); the long usertests preempt test is expected to hang under SJF, so I did not run usertests under SJF. Under PRIORITYRR a busy process at a higher level than the shell would delay the shell until it finishes.
- **Complex commands:** normal shell pipelines work with the new schedulers. The Part 1 commands (sort, grep) and the Part 2 syscall behave the same under all three.

Resources used

- **Claude (Anthropic AI assistant):** used to understand xv6's scheduler, trap and syscall code, and to help draft the code for sort, the grep flags, ticks_running, the SJF and priority schedulers and the test programs.
- **xv6 source** (proc.c, trap.c, sysproc.c, grep.c, cat.c, sh.c) and the xv6 book chapters on traps, scheduling and system calls: read as reference.

Part 1: Sort and Grep

Building xv6 with sort added.

```
ankrit@DESKTOP-IGT6P4O:~/xv6-public$ make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*.o *.d *.asm *.sym vectors.S bootblock entryother \
initcode initcode.out kernel xv6.img fs.img kernelmemfs \
xv6memfs.img mkfs .gdbinit \
_cat _echo _forktest _grep _init _kill _ln _ls _mkdir _rm _sh _stres
sfs _usertests _wc _zombie _hello _sleep _sort
ankrit@DESKTOP-IGT6P4O:~/xv6-public$ make
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD
 -ggdb -m32 -Werror -fno-omit-frame-pointer -Wno-array-bounds -Wno-i
nfinite-recursion -fno-stack-protector -fno-pie -no-pie -fno-pic -O
-nostdinc -I. -c bootmain.c
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD
```

```
dd if=/dev/zero of=xv6.img count=10000
10000+0 records in
10000+0 records out
5120000 bytes (5.1 MB, 4.9 MiB) copied, 0.0144303 s, 366 MB/s
dd if=bootblock of=xv6.img conv=notrunc
1+0 records in
1+0 records out
512 bytes copied, 0.000285051 s, 512 kB/s
dd if=kernel of=xv6.img seek=1 conv=notrunc
395+1 records in
395+1 records out
202516 bytes (203 kB, 198 KiB) copied, 0.000946641 s, 203 MB/s
ankrit@DESKTOP-IGT6P4O:~/xv6-public$
```

Two text files, os611.txt and nums.txt and their content.

```
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logs
init: starting sh
$ ls
README          2 2 3405
os611.txt       2 3 149
nums.txt        2 4 14
cat             2 5 16060
echo            2 6 14952
forktest        2 7 9008
grep            2 8 18912
init            2 9 15560
kill            2 10 15028
ln              2 11 14896
ls              2 12 18080
mkdir           2 13 15024
rm              2 14 15008
sh              2 15 29016
stressfs        2 16 15852
usertests       2 17 63736
wc              2 18 16360
zombie          2 19 14596
hello           2 20 14768
sleep           2 21 14888
sort            2 22 18772
console         3 23 0
$ cat os611.txt
I understand Operating system.
I understand Operating system.
I understand Operating system.
I love to work on OS.
I love to work on OS.
Thanks xv6.
$
$
$ cat nums.txt
30
5
100
5
20
$ sort os611.txt
```

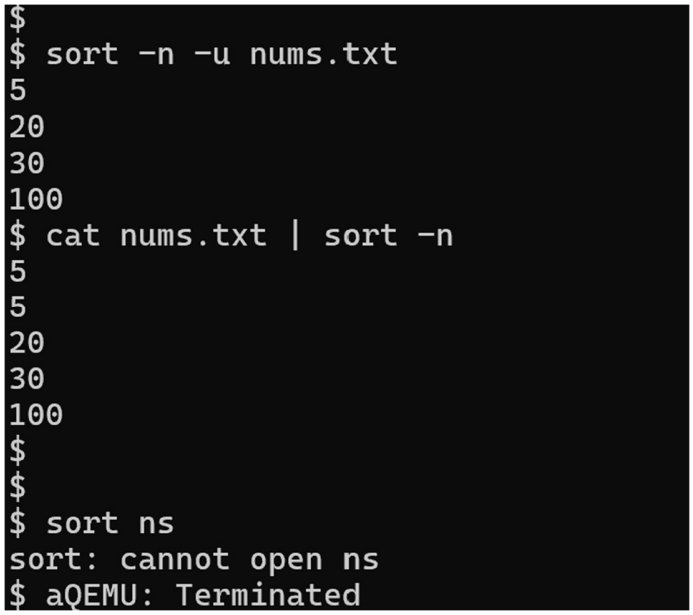
Test output: Sort

```
$ sort os611.txt
I love to work on OS.
I love to work on OS.
I understand Operating system.
I understand Operating system.
I understand Operating system.
Thanks xv6.
$
$
$
$ sort os611.txt
I love to work on OS.
I love to work on OS.
I understand Operating system.
I understand Operating system.
I understand Operating system.
Thanks xv6.
$
$
$
$ sort nums.txt
100
20
30
5
5
$
$
```

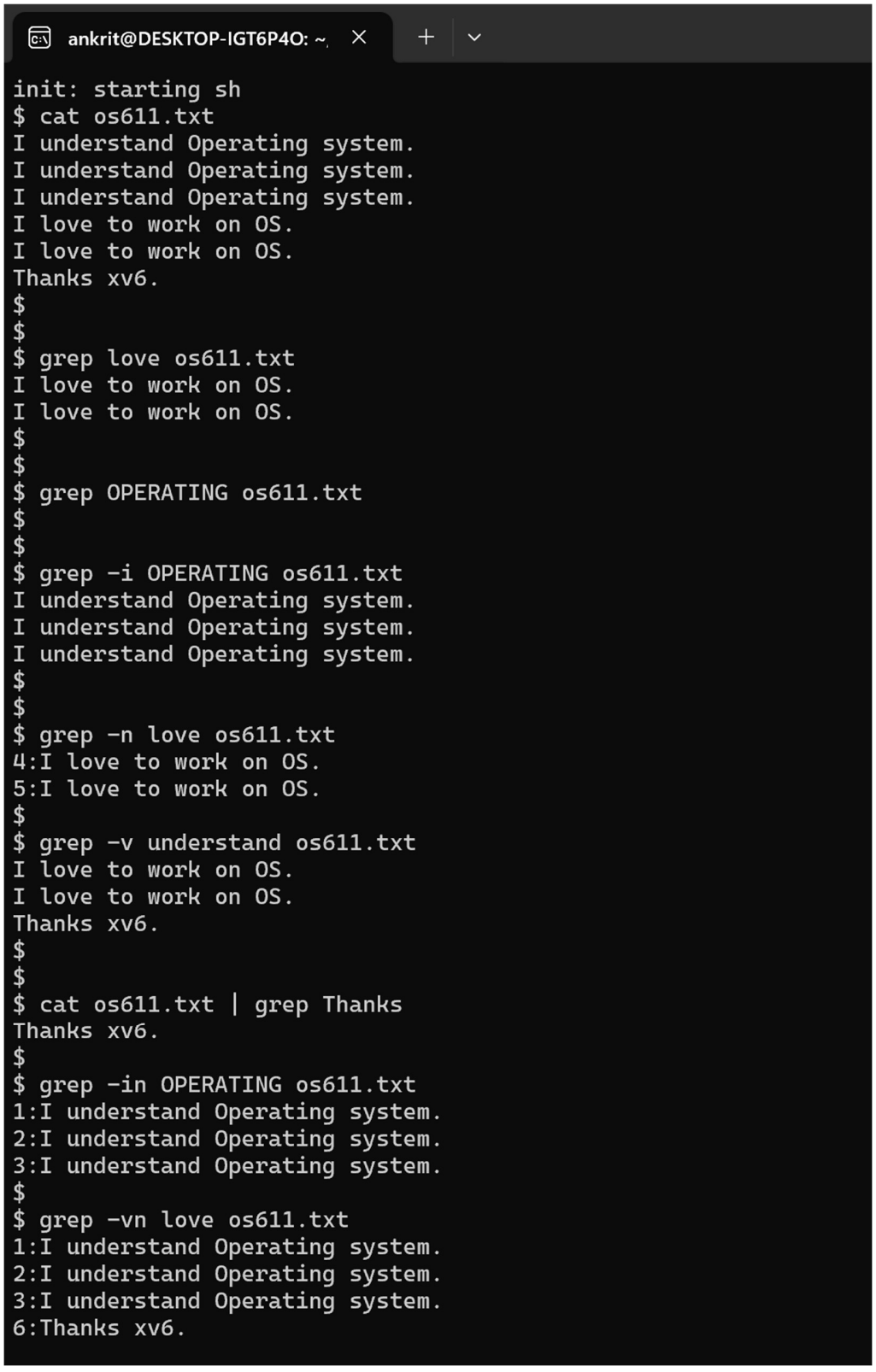
```
$
$ sort -n nums.txt
5
5
20
30
100
$
$
$ sort -r os611.txt
Thanks xv6.
I understand Operating system.
I understand Operating system.
I understand Operating system.
I love to work on OS.
I love to work on OS.
$
$
$
$ sort -u os611.txt
I love to work on OS.
I understand Operating system.
Thanks xv6.
$
$
$ cat os611.txt | sort
I love to work on OS.
I love to work on OS.
I understand Operating system.
I understand Operating system.
I understand Operating system.
Thanks xv6.
$
$
```

Some extra checks.

```
$ sort -r -n nums.txt
100
30
20
5
5
$ sort
```

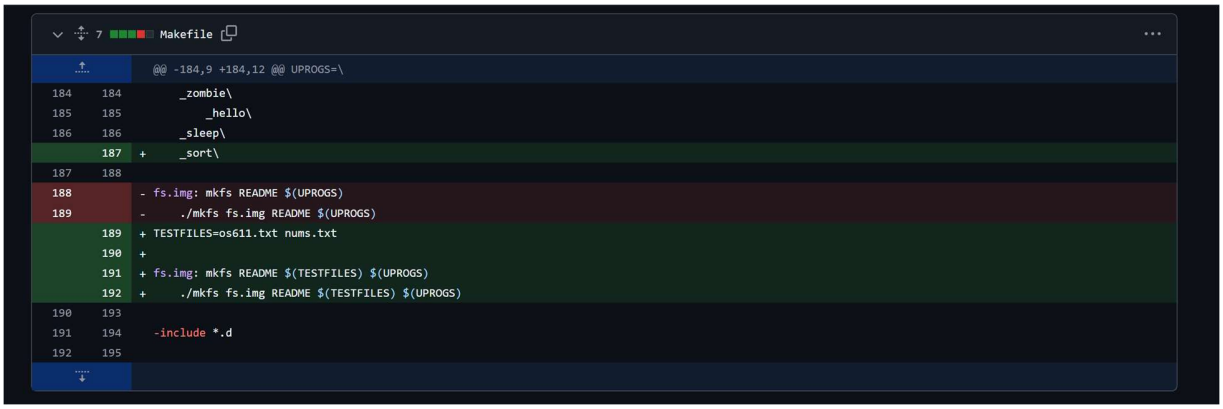

$
$ sort -n -u nums.txt
5
20
30
100
$ cat nums.txt | sort -n
5
5
20
30
100
$
$
$ sort ns
sort: cannot open ns
$ aQEMU: Terminated

Test Output: Grep



Code Changes

Makefile

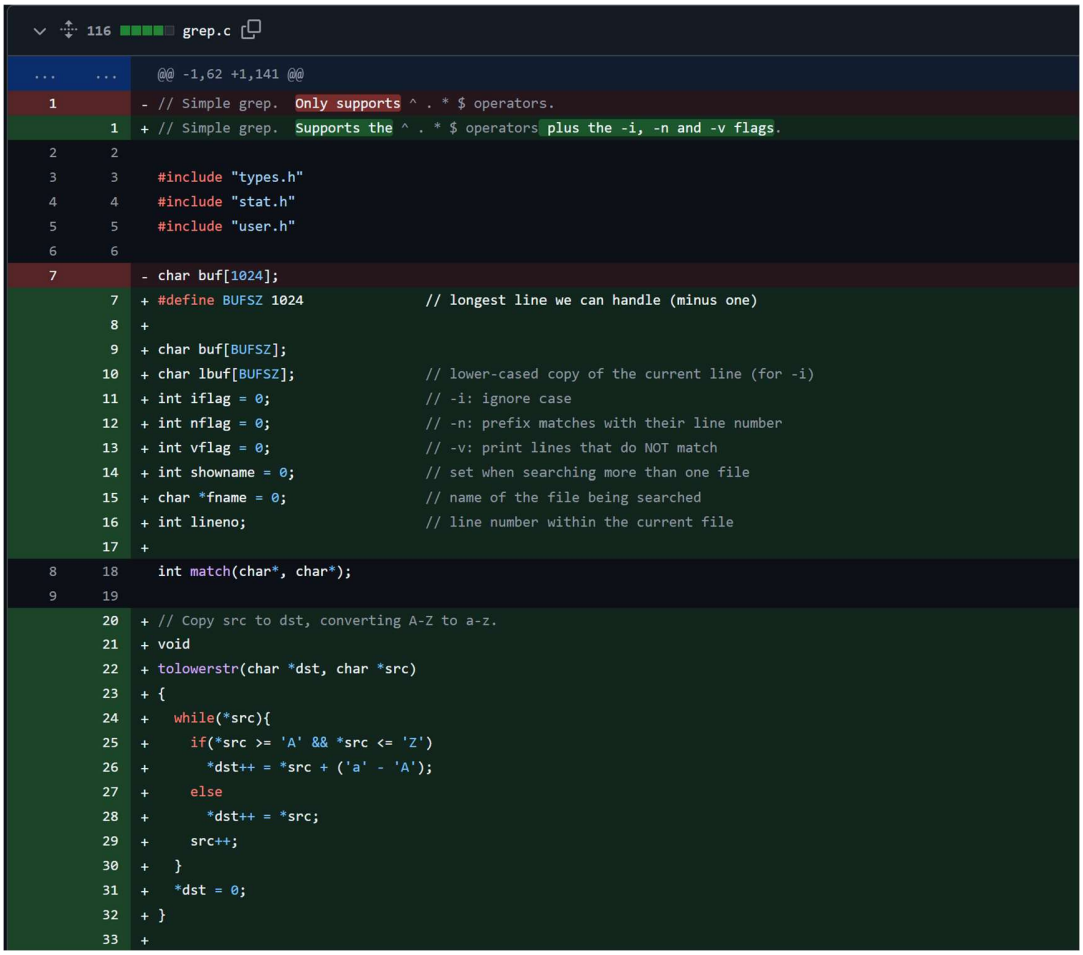


```
Makefile
@@ -184,9 +184,12 @@ UPROGS=\
    _zombie\
    _hello\
    _sleep\
+   _sort\

- fs.img: mkfs README $(UPROGS)
-     ./mkfs fs.img README $(UPROGS)
+ TESTFILES=os611.txt nums.txt
+
+ fs.img: mkfs README $(TESTFILES) $(UPROGS)
+     ./mkfs fs.img README $(TESTFILES) $(UPROGS)

 -include *.d
```

Grep.c



```
grep.c
@@ -1,62 +1,141 @@
- // Simple grep.  Only supports ^ . * $ operators.
+ // Simple grep.  Supports the ^ . * $ operators plus the -i, -n and -v flags.

 #include "types.h"
 #include "stat.h"
 #include "user.h"

- char buf[1024];
+ #define BUFSZ 1024              // longest line we can handle (minus one)
+
+ char buf[BUFSZ];
+ char lbuf[BUFSZ];               // lower-cased copy of the current line (for -i)
+ int iflag = 0;                  // -i: ignore case
+ int nflag = 0;                  // -n: prefix matches with their line number
+ int vflag = 0;                  // -v: print lines that do NOT match
+ int showname = 0;               // set when searching more than one file
+ char *fname = 0;                // name of the file being searched
+ int lineno;                     // line number within the current file
+
 int match(char*, char*);

+ // Copy src to dst, converting A-Z to a-z.
+ void
+ tolowerstr(char *dst, char *src)
+ {
+   while(*src){
+     if(*src >= 'A' && *src <= 'Z')
+       *dst++ = *src + ('a' - 'A');
+     else
+       *dst++ = *src;
+     src++;
+   }
+   *dst = 0;
+ }
+
```

```
+ // Handle one complete line (already NUL-terminated).
+ void
+ doline(char *pattern, char *line)
+ {
+   int hit;
+
+   lineno++;
+   if(iflag){
+     tolowerstr(lbuf, line);       // match against the lower-cased copy...
+     hit = match(pattern, lbuf);
+   } else {
+     hit = match(pattern, line);
+   }
+   if(vflag)
+     hit = !hit;
+   if(!hit)
+     return;
+   if(showname)
+     printf(1, "%s:", fname);
+   if(nflag)
+     printf(1, "%d:", lineno);
+   printf(1, "%s\n", line);        // ...but always print the original line
+ }
+
  void
  grep(char *pattern, int fd)
  {
    int n, m;
    char *p, *q;

+   lineno = 0;
    m = 0;
-   while((n = read(fd, buf+m, sizeof(buf)-m-1)) > 0){
+   while((n = read(fd, buf+m, BUFSZ-m-1)) > 0){
      m += n;
      buf[m] = '\0';
      p = buf;
      while((q = strchr(p, '\n')) != 0){
```

```
21  70          while((q = strchr(p, '\n')) != 0){
22  71            *q = 0;
23      -         if(match(pattern, p)){
24      -           *q = '\n';
25      -           write(1, p, q+1 - p);
26      -         }
    72  +         doline(pattern, p);
27  73            p = q+1;
28  74          }
29      -       if(p == buf)
    75  +       // Keep any unfinished line at the front of buf for the next read.
    76  +       m -= p - buf;
    77  +       memmove(buf, p, m);
    78  +       if(m >= BUFSZ-1){             // a single line longer than the buffer
    79  +         buf[m] = '\0';
    80  +         doline(pattern, buf);
30  81            m = 0;
31      -       if(m > 0){
32      -         m -= p - buf;
33      -         memmove(buf, p, m);
34  82          }
35  83        }
    84  +     if(m > 0){                      // last line had no trailing newline
    85  +       buf[m] = '\0';
    86  +       doline(pattern, buf);
    87  +     }
36  88    }
37  89
38  90    int
39  91    main(int argc, char *argv[])
40  92    {
41      -   int fd, i;
42      -   char *pattern;
    93  +   int fd, i, first;
    94  +   char *pattern, *c;
    95  +   char lpat[BUFSZ];
```

```
96  +
97  +   // Leading arguments like -i, -n, -v, or clusters such as -in, are flags.
98  +   for(first = 1; first < argc && argv[first][0] == '-' && argv[first][1] != 0; first++){
99  +     for(c = argv[first] + 1; *c; c++){
100 +       if(*c == 'i')
101 +         iflag = 1;
102 +       else if(*c == 'n')
103 +         nflag = 1;
104 +       else if(*c == 'v')
105 +         vflag = 1;
106 +       else{
107 +         printf(2, "grep: unknown option -%c\n", *c);
108 +         exit();
109 +       }
110 +     }
111 +   }
43  112
44      -   if(argc <= 1){
45      -     printf(2, "usage: grep pattern [file ...]\n");
113 +   if(first >= argc){
114 +     printf(2, "usage: grep [-inv] pattern [file ...]\n");
115 +     exit();
116 +   }
117 +   pattern = argv[first++];
118 +   if(strlen(pattern) >= BUFSZ){
119 +     printf(2, "grep: pattern too long\n");
46  120       exit();
47  121     }
48      -   pattern = argv[1];
122 +   if(iflag){
123 +     tolowerstr(lpat, pattern);    // lower-case the pattern once, up front
124 +     pattern = lpat;
125 +   }
49  126
50      -   if(argc <= 2){
127 +   if(first >= argc){              // no file given: read standard input
51  128       grep(pattern, 0);
52  129       exit();
53  130     }
```

```
54  131
55      -   for(i = 2; i < argc; i++){
        132 +   showname = (argc - first > 1);
        133 +   for(i = first; i < argc; i++){
56  134         if((fd = open(argv[i], 0)) < 0){
57  135           printf(1, "grep: cannot open %s\n", argv[i]);
58  136           exit();
59  137         }
        138 +     fname = argv[i];
60  139         grep(pattern, fd);
61  140         close(fd);
62  141       }
          @@ -104,4 +183,3 @@ int matchstar(int c, char *re, char *text)
104 183     }while(*text!='\0' && (*text++==c || c=='.'));
105 184     return 0;
106 185   }
107     -
```

5 nums.txt

Text files

5 nums.txt

```
...  ...   @@ -0,0 +1,5 @@
1   + 30
2   + 5
3   + 100
4   + 5
5   + 20
```

6 os611.txt

```
...  ...   @@ -0,0 +1,6 @@
1   + I understand Operating system.
2   + I understand Operating system.
3   + I understand Operating system.
4   + I love to work on OS.
5   + I love to work on OS.
6   + Thanks xv6.
```

Sort. C

100 sort.c

```
@@ -0,0 +1,100 @@
+ #include "types.h"
+ #include "stat.h"
+ #include "user.h"
+
+ #define MAXBUF  65536
+ #define MAXLINES 2048
+
+ char buf[MAXBUF];
+ char *lines[MAXLINES];
+ int nlines = 0;
+
+ int rflag = 0, nflag = 0, uflag = 0;
+
+ int
+ mystrcmp(char *a, char *b, int numeric)
+ {
+   if(numeric)
+     return atoi(a) - atoi(b);
+   return strcmp(a, b);
+ }
+
+ // Simple insertion sort over the array of line pointers.
+ // Fine for the sizes this assignment tests; swaps pointers, not text.
+ void
+ sortlines(void)
+ {
+   int i, j;
+   char *key;
+
+   for(i = 1; i < nlines; i++){
+     key = lines[i];
+     j = i - 1;
+     while(j >= 0 && mystrcmp(lines[j], key, nflag) * (rflag ? -1 : 1) > 0){
+       lines[j+1] = lines[j];
+       j--;
+     }
```

```
+     }
+     lines[j+1] = key;
+   }
+ }
+
+ int
+ main(int argc, char *argv[])
+ {
+   int fd = 0;     // default: stdin
+   char *fname = 0;
+   int i, n, total = 0;
+
+   for(i = 1; i < argc; i++){
+     if(strcmp(argv[i], "-r") == 0)
+       rflag = 1;
+     else if(strcmp(argv[i], "-n") == 0)
+       nflag = 1;
+     else if(strcmp(argv[i], "-u") == 0)
+       uflag = 1;
+     else
+       fname = argv[i];        // the one non-flag argument is the filename
+   }
+
+   if(fname){
+     if((fd = open(fname, 0)) < 0){
+       printf(2, "sort: cannot open %s\n", fname);
+       exit();
+     }
+   }
+
+   // Read the whole input into buf.
+   while((n = read(fd, buf + total, MAXBUF - total)) > 0)
+     total += n;
+   if(fname)
+     close(fd);
+
+   if(total >= MAXBUF){
+     printf(2, "sort: input too large\n");
+     exit();
+   }
+   buf[total] = 0;
+
+   // Split into lines
```

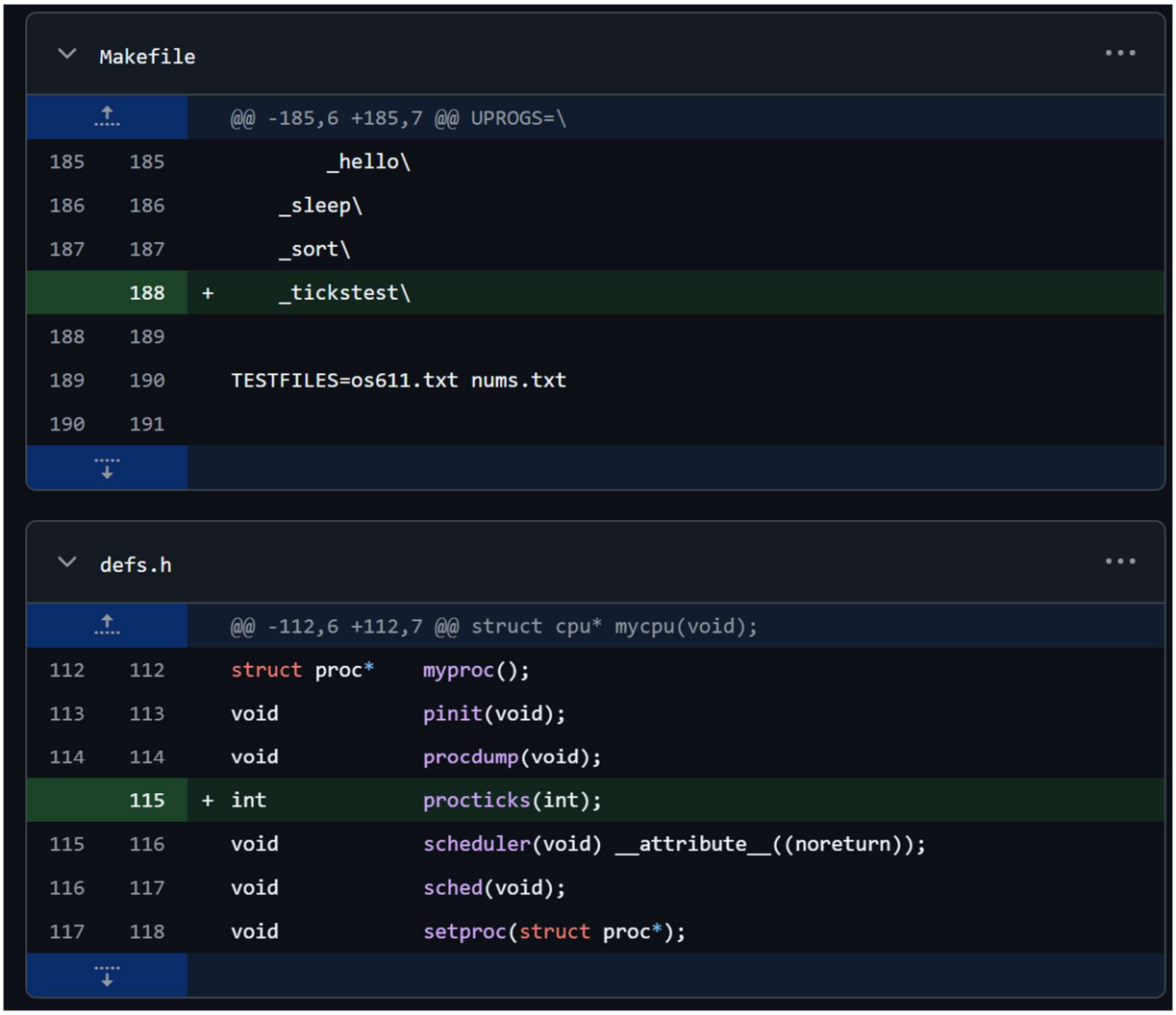
```
78  +   // Split into lines.
79  +   lines[nlines++] = buf;
80  +   for(i = 0; i < total; i++){
81  +     if(buf[i] == '\n'){
82  +       buf[i] = 0;
83  +       if(i + 1 < total && nlines < MAXLINES)
84  +         lines[nlines++] = buf + i + 1;
85  +     }
86  +   }
87  +   // Drop a trailing empty "line" caused by a final newline.
88  +   if(nlines > 0 && lines[nlines-1][0] == 0)
89  +     nlines--;
90  +
91  +   sortlines();
92  +
93  +   for(i = 0; i < nlines; i++){
94  +     if(uflag && i > 0 && strcmp(lines[i], lines[i-1]) == 0)
95  +       continue;               // skip duplicate of the previous (sorted) line
96  +     printf(1, "%s\n", lines[i]);
97  +   }
98  +
99  +   exit();
100 + }
```

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal informat

Part 2: Ticks Running

```
$ tickstest
Test 1: the calling process (pid 4)
  before spinning:     1 ticks
  after spinning ~50:  52 ticks (expect about 50)
Test 2: another process
  child 5 just after fork: 0 ticks (expect 0)
  child 5 after it ran:    29 ticks (expect about 30)
  child 5 after wait():    -1 (expect -1, it is gone)
Test 3: processes that mostly sleep
  init (pid 1): 3 ticks
  sh   (pid 2): 1 ticks
Test 4: pids that do not exist
  pid 9999: -1 (expect -1)
  pid -1:   -1 (expect -1)
  pid 0:    -1 (expect -1)
$
$
$ hello
Hello Xv6!
Hello from the kernel!
$ forktest
fork test
fork test OK
$
```

Code Changes:



Makefile

```
@@ -185,6 +185,7 @@ UPROGS=\
185 185         _hello\
186 186         _sleep\
187 187         _sort\
    188 +       _tickstest\
188 189
189 190     TESTFILES=os611.txt nums.txt
190 191
```

defs.h

```
@@ -112,6 +112,7 @@ struct cpu* mycpu(void);
112 112     struct proc*    myproc();
113 113     void            pinit(void);
114 114     void            procdump(void);
    115 +   int             procticks(int);
115 116     void            scheduler(void) __attribute__((noreturn));
116 117     void            sched(void);
117 118     void            setproc(struct proc*);
```

proc.c

```
                p->pid = nextpid++;
91  +           p->run_ticks = 0;
91  92
92  93          release(&ptable.lock);
93  94
                @@ -497,6 +498,25 @@ kill(int pid)
497 498       }
498 499
499 500       //PAGEBREAK: 36
501 +   // Return the number of timer ticks process pid has spent RUNNING,
502 +   // or -1 if no process with that pid is in the process table.
503 +   int
504 +   procticks(int pid)
505 +   {
506 +     struct proc *p;
507 +     int t = -1;
508 +
509 +     acquire(&ptable.lock);
510 +     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
511 +       if(p->state != UNUSED && p->pid == pid){
512 +         t = p->run_ticks;
513 +         break;
514 +       }
515 +     }
516 +     release(&ptable.lock);
517 +     return t;
518 +   }
```

proc.h

```
49  49      struct file *ofile[NOFILE];  // Open files
50  50      struct inode *cwd;           // Current directory
51  51      char name[16];               // Process name (debugging)
    52  +   int run_ticks;               // Timer ticks spent in the RUNNING state
52  53    };
53  54
54  55    // Process memory is laid out contiguously, low addresses first:
```

syscall.c

```
      @@ -104,6 +104,7 @@ extern int sys_wait(void);
104 104   extern int sys_write(void);
105 105   extern int sys_uptime(void);
106 106   extern int sys_hello(void);
    107 + extern int sys_ticks_running(void);
107 108
108 109   static int (*syscalls[])(void) = {
109 110   [SYS_fork]    sys_fork,
      @@ -128,6 +129,7 @@ static int (*syscalls[])(void) = {
128 129   [SYS_mkdir]   sys_mkdir,
129 130   [SYS_close]   sys_close,
130 131   [SYS_hello]   sys_hello,
    132 + [SYS_ticks_running] sys_ticks_running,
131 133   };
132 134
133 135   void
```

syscall.h

```
@@ -21,3 +21,4 @@
21  21  #define SYS_mkdir  20
22  22  #define SYS_close  21
23  23  #define SYS_hello  22
    24 + #define SYS_ticks_running 23
```

sysproc.c

```
@@ -98,3 +98,13 @@ sys_hello(void)
98  98    cprintf("Hello from the kernel!\n");
99  99    return 0;
100 100   }
    101 +
    102 + int
    103 + sys_ticks_running(void)
    104 + {
    105 +   int pid;
    106 +
    107 +   if(argint(0, &pid) < 0)
    108 +     return -1;
    109 +   return procticks(pid);
    110 + }
```

tickstest.c

```
+ #include "types.h"
+ #include "stat.h"
+ #include "user.h"
+
+ // Burn CPU (without sleeping) for about n ticks of wall-clock time.
+ void
+ spin(int n)
+ {
+   int start = uptime();
+
+   while(uptime() - start < n)
+     ;
+ }
+
+ int
+ main(int argc, char *argv[])
+ {
+   int pid, me;
+
+   me = getpid();
+
+   printf(1, "Test 1: the calling process (pid %d)\n", me);
+   printf(1, "  before spinning:     %d ticks\n", ticks_running(me));
+   spin(50);
+   printf(1, "  after spinning ~50:  %d ticks (expect about 50)\n",
  ticks_running(me));
+
+   printf(1, "Test 2: another process\n");
```

tickstest.c

```
26  +
27  +   printf(1, "Test 2: another process\n");
28  +   pid = fork();
29  +   if(pid < 0){
30  +     printf(2, "tickstest: fork failed\n");
31  +     exit();
32  +   }
33  +   if(pid == 0){
34  +     spin(30);                  // child burns about 30 ticks, then exits
35  +     exit();
36  +   }
37  +   printf(1, "  child %d just after fork: %d ticks (expect 0)\n", pid,
      ticks_running(pid));
38  +   sleep(40);                   // let the child finish; it stays a zombie until wait()
39  +   printf(1, "  child %d after it ran:    %d ticks (expect about 30)\n", pid,
      ticks_running(pid));
40  +   wait();
41  +   printf(1, "  child %d after wait():    %d (expect -1, it is gone)\n", pid,
      ticks_running(pid));
42  +
43  +   printf(1, "Test 3: processes that mostly sleep\n");
44  +   printf(1, "  init (pid 1): %d ticks\n", ticks_running(1));
45  +   printf(1, "  sh   (pid 2): %d ticks\n", ticks_running(2));
46  +
47  +   printf(1, "Test 4: pids that do not exist\n");
48  +   printf(1, "  pid 9999: %d (expect -1)\n", ticks_running(9999));
49  +   printf(1, "  pid -1:   %d (expect -1)\n", ticks_running(-1));
50  +   printf(1, "  pid 0:    %d (expect -1)\n", ticks_running(0));
51  +
52  +   exit();
53  + }
```

trap.c

```
@@ -54,6 +54,9 @@ trap(struct trapframe *tf)
54 54          wakeup(&ticks);
55 55          release(&tickslock);
56 56        }
   57 +      // Charge this tick to the process that was running, if any.
   58 +      if(myproc() && myproc()->state == RUNNING)
   59 +        myproc()->run_ticks++;
57 60        lapiceoi();
58 61        break;
59 62      case T_IRQ0 + IRQ_IDE:
```

user.h

```
@@ -24,6 +24,7 @@ char* sbrk(int);
24 24   int sleep(int);
25 25   int uptime(void);
26 26   int hello(void);
   27 + int ticks_running(int);
27 28
28 29   // ulib.c
29 30   int stat(const char*, struct stat*);
```

usys.S

```
@@ -30,3 +30,4 @@ SYSCALL(sbrk)
30 30   SYSCALL(sleep)
31 31   SYSCALL(uptime)
32 32   SYSCALL(hello)
   33 + SYSCALL(ticks_running)
```

Part 3: Simple scheduler, SJF

Under Default scheduler, all children finish at the same time.

```
$
$ sjftest
self   pid 5: sjf_job_length = 40
bogus  pid 9999: sjf_job_length = -1 (expect -1)
children (predicted lengths): 6:71 7:99 8:37 9:67 10:45
  finished: pid 6  pre  finished: pid 7  predicted length   finished: pid 83
ed length 37  at ed length 67  at tick 715
dicted length 71  at tick 712
tick 714
  finished: pid 10  predicted length 45  at tick 720
after wait, pid 6: sjf_job_length = -1 (expect -1)
$
```

Output of **timeit stressfs**

```
$
$ timeit stressfs
stressfs starting
write write 1
write 2
0
write 3
write 4
read
read
read
read
read
timeit: stressfs took 47 ticks
$
$
```

Output of **timeit ls**

```
$
$
$ timeit ls
README          2 2 3405
os611.txt       2 3 149
nums.txt        2 4 14
cat             2 5 16228
echo            2 6 15120
forktest        2 7 9176
grep            2 8 22052
init            2 9 15728
kill            2 10 15196
ln              2 11 15060
ls              2 12 18248
mkdir           2 13 15192
rm              2 14 15172
sh              2 15 29184
stressfs        2 16 16020
usertests       2 17 63904
wc              2 18 16528
zombie          2 19 14760
hello           2 20 14936
sleep           2 21 15056
sort            2 22 18940
tickstest       2 23 16868
sjftest         2 24 17040
timeit          2 25 15544
priotest        2 26 19324
console         3 27 0
stressfs2       2 28 10240
stressfs0       2 29 10240
stressfs1       2 30 10240
stressfs3       2 31 10240
stressfs4       2 32 10240
timeit: ls took 41 ticks
$
```

SJF

make xv6.img SCHEDULER=[SJF]: -DSJF is passed to the compiler

Note: Could not capture the screenshot with qemu-nox, so used a grep handler to build the image and grab the interested lines.

```
ankrit@DESKTOP-IGT6P4O:~/xv6-public$ make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*.o *.d *.asm *.sym vectors.S bootblock entryother \
initcode initcode.out kernel xv6.img fs.img kernelmemfs \
xv6memfs.img mkfs .gdbinit \
_cat _echo _forktest _grep _init _kill _ln _ls _mkdir _rm _sh _stressfs _usertests _wc _zombie _hello _sleep _sort _tickstest _sjftest _timeit _priotest  .sched_*
ankrit@DESKTOP-IGT6P4O:~/xv6-public$
ankrit@DESKTOP-IGT6P4O:~/xv6-public$ make xv6.img 'SCHEDULER=[SJF]' 2>&1 | grep -oE "\-D[A-Z]+ -DRR_QUANTUM=[0-9]+ +-c -o [a-z]+\.o [a-z]+\.c" | grep -E " proc\.o| tr"
-DSJF -DRR_QUANTUM=5   -c -o proc.o proc.c
-DSJF -DRR_QUANTUM=5   -c -o trap.o trap.c
```

Output of **sjftest under sjf**

```
$
$ sjftest
self   pid 6: sjf_job_length = 71
bogus  pid 9999: sjf_job_length = -1 (expect -1)
children (predicted lengths): 7:99 8:37 9:67 10:45 11:12
  finished: pid 11  predicted length 12  at tick 400
  finished: pid 8  predicted length 37  at tick 432
  finished: pid 10  predicted length 45  at tick 463
  finished: pid 9  predicted length 67  at tick 495
  finished: pid 7  predicted length 99  at tick 527
after wait, pid 7: sjf_job_length = -1 (expect -1)
$
$
```

Output of **timeit stressfs**

```
$ timeit stressfs
stressfs starting
write 0
write 1
write 3
read
read
write 2
write 4
read
read
read
timeit: stressfs took 44 ticks
$
```

Output of **timeit ls**

```
$
$ timeit ls
README          2 2 3405
os611.txt       2 3 149
nums.txt        2 4 14
cat             2 5 16228
echo            2 6 15120
forktest        2 7 9176
grep            2 8 22052
init            2 9 15728
kill            2 10 15196
ln              2 11 15060
ls              2 12 18248
mkdir           2 13 15192
rm              2 14 15172
sh              2 15 29184
stressfs        2 16 16020
usertests       2 17 63904
wc              2 18 16528
zombie          2 19 14760
hello           2 20 14936
sleep           2 21 15056
sort            2 22 18940
tickstest       2 23 16868
sjftest         2 24 17040
timeit          2 25 15544
priotest        2 26 19324
console         3 27 0
stressfs0       2 28 10240
stressfs1       2 29 10240
stressfs2       2 30 10240
stressfs3       2 31 10240
stressfs4       2 32 10240
timeit: ls took 41 ticks
$
$
$
```

Code Changes:

Makefile

Makefile +23 -1

```
+ # Pick the scheduler when you build:
+ #     make qemu-nox SCHEDULER=DEFAULT      (the original xv6 scheduler; default)
+ #     make qemu-nox SCHEDULER=SJF          (Shortest Job First)
+ # The name reaches the compiler as -D<name>, which proc.c and trap.c test.
+ SCHEDULER ?= DEFAULT
+ SCHED := $(strip $(subst ],,$(subst [,,$(SCHEDULER))))
+ ifneq ($(filter $(SCHED),DEFAULT SJF),$(SCHED))
+ $(error Unknown SCHEDULER "$(SCHEDULER)": use DEFAULT or SJF)
+ endif
+ CFLAGS += -D$(SCHED)
+
+ # make cannot see that CFLAGS changed, so remember which scheduler the objects
+ # were built for and rebuild the ones that depend on it when it changes.
+ SCHED_STAMP := .sched_$(SCHED)
+ $(SCHED_STAMP):
+     @rm -f .sched_*
+     @touch $@
+ proc.o trap.o: $(SCHED_STAMP)
+
 xv6memfs.img: bootblock kernelmemfs
     dd if=/dev/zero of=xv6memfs.img count=10000
     dd if=bootblock of=xv6memfs.img conv=notrunc
@@ -186,6 +206,8 @@ UPROGS=\
     _sleep\
     _sort\
     _tickstest\
+     _sjftest\
+     _timeit\

 TESTFILES=os611.txt nums.txt

@@ -199,7 +221,7 @@ clean:
     *.o *.d *.asm *.sym vectors.S bootblock entryother \
     initcode initcode.out kernel xv6.img fs.img kernelmemfs \
     xv6memfs.img mkfs .gdbinit \
-     $(UPROGS)
+     $(UPROGS) .sched_*
```

Proc.c changes

proc.c +61 -1

```
21  21  static void wakeup1(void *chan);
22  22
    23  + // Predicted job lengths for the SJF scheduler come from a simple linear
    24  + // congruential generator, so they vary from process to process but are
    25  + // repeatable from boot to boot.  Callers hold ptable.lock.
    26  + static uint sjf_seed = SJF_SEED;
    27  +
    28  + static int
    29  + next_job_len(void)
    30  + {
    31  +   sjf_seed = sjf_seed * SJF_LCG_A + SJF_LCG_C;
    32  +   return SJF_MINLEN + (sjf_seed >> 16) % (SJF_MAXLEN - SJF_MINLEN + 1);
    33  + }
    34  +
23  35  void
24  36  pinit(void)
25  37  {
        @@ -89,6 +101,7 @@ allocproc(void)
89  101   p->state = EMBRYO;
90  102   p->pid = nextpid++;
91  103   p->run_ticks = 0;
    104 +   p->job_len = next_job_len();
92  105
93  106   release(&ptable.lock);
94  107
        @@ -324,15 +337,42 @@ void
324 337  scheduler(void)
325 338  {
326 339    struct proc *p;
    340 + #ifdef SJF
    341 +   struct proc *best;
    342 + #endif
```

proc.c +61 -1

```
327 343       struct cpu *c = mycpu();
328 344       c->proc = 0;
329 345
330 346       for(;;){
331 347         // Enable interrupts on this processor.
332 348         sti();
333 349
334     -       // Loop over process table looking for process to run.
335 350         acquire(&ptable.lock);
    351 + #ifdef SJF
    352 +       // Shortest Job First: of all RUNNABLE processes, run the one with the
    353 +       // smallest predicted job length (ties go to the lowest table slot).
    354 +       // trap.c does not preempt on timer ticks in this mode, so the chosen
    355 +       // process keeps the CPU until it sleeps or exits.
    356 +       best = 0;
    357 +       for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    358 +         if(p->state != RUNNABLE)
    359 +           continue;
    360 +         if(best == 0 || p->job_len < best->job_len)
    361 +           best = p;
    362 +       }
    363 +       if(best != 0){
    364 +         p = best;
    365 +         c->proc = p;
    366 +         switchuvm(p);
    367 +         p->state = RUNNING;
    368 +
    369 +         swtch(&(c->scheduler), p->context);
    370 +         switchkvm();
    371 +
    372 +         c->proc = 0;
    373 +       }
    374 + #else
    375 +       // Default xv6 scheduler: loop over process table looking for process to run.
336 376         for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
337 377           if(p->state != RUNNABLE)
```

proc.c +61

```
@@ -351,6 +391,7 @@ scheduler(void)
351 391          // It should have changed its p->state before coming back.
352 392          c->proc = 0;
353 393        }
    394 +    #endif
354 395        release(&ptable.lock);
355 396
356 397      }
@@ -517,6 +558,25 @@ procticks(int pid)
517 558    return t;
518 559  }
519 560
    561 + // Return the predicted job length of process pid, or -1 if there is no such
    562 + // process or it is not in the scheduling queue (RUNNABLE or RUNNING).
    563 + int
    564 + procjoblen(int pid)
    565 + {
    566 +   struct proc *p;
    567 +   int len = -1;
    568 +
    569 +   acquire(&ptable.lock);
    570 +   for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    571 +     if(p->pid == pid && (p->state == RUNNABLE || p->state == RUNNING)){
    572 +       len = p->job_len;
    573 +       break;
    574 +     }
    575 +   }
    576 +   release(&ptable.lock);
    577 +   return len;
    578 + }
    579 +
520 580  // Print a process listing to console.  For debugging.
521 581  // Runs when user types ^P on console.
522 582  // No lock to avoid wedging a stuck machine further.
```

Sjftest.c

sjftest.c +64

```
@@ -0,0 +1,64 @@
+ // sjftest: shows the predicted job lengths handed out by the SJF scheduler
+ // and the order in which a batch of equally busy children finishes.
+ //   usage: sjftest [children]        (default 5)
+ // Under SJF the children finish shortest-predicted-job first.
+ // Under the default scheduler they take turns and finish in about pid order.
+ #include "types.h"
+ #include "stat.h"
+ #include "user.h"
+
+ #define DEFAULT_KIDS 5
+ #define MAX_KIDS     12
+ #define WORK_TICKS   30   // each child burns this many ticks of CPU
+
+ static void
+ spin(int n)
+ {
+   int start = uptime();
+   while(uptime() - start < n)
+     ;
+ }
+
+ int
+ main(int argc, char *argv[])
+ {
+   int kids = DEFAULT_KIDS;
+   int pids[MAX_KIDS];
+   int i, me = getpid();
+
+   if(argc > 1)
+     kids = atoi(argv[1]);
+   if(kids < 1 || kids > MAX_KIDS){
+     printf(1, "usage: sjftest [1-%d]\n", MAX_KIDS);
+     exit();
+   }
```

```
35 +
36 +   printf(1, "self   pid %d: sjf_job_length = %d\n", me, sjf_job_length(me));
37 +   printf(1, "bogus  pid 9999: sjf_job_length = %d (expect -1)\n", sjf_job_length(9999));
38 +
39 +   for(i = 0; i < kids; i++){
40 +     int pid = fork();
41 +     if(pid < 0){
42 +       printf(1, "fork failed\n");
43 +       exit();
44 +     }
45 +     if(pid == 0){
46 +       spin(WORK_TICKS);
47 +       printf(1, "  finished: pid %d  predicted length %d  at tick %d\n",
48 +              getpid(), sjf_job_length(getpid()), uptime());
49 +       exit();
50 +     }
51 +     pids[i] = pid;
52 +   }
53 +
54 +   printf(1, "children (predicted lengths):");
55 +   for(i = 0; i < kids; i++)
56 +     printf(1, " %d:%d", pids[i], sjf_job_length(pids[i]));
57 +   printf(1, "\n");
58 +
59 +   for(i = 0; i < kids; i++)
60 +     wait();
61 +   printf(1, "after wait, pid %d: sjf_job_length = %d (expect -1)\n",
62 +          pids[0], sjf_job_length(pids[0]));
63 +   exit();
64 + }
```

Trap.c

trap.c +4

```
@@ -105,9 +105,13 @@ trap(struct trapframe *tf)
105 105
106 106     // Force process to give up CPU on clock tick.
107 107     // If interrupts were on while locks held, would need to check nlock.
    108 +   // SJF is non-preemptive, so it skips this: a process keeps the CPU until
    109 +   // it sleeps or exits.
    110 + #ifndef SJF
108 111     if(myproc() && myproc()->state == RUNNING &&
109 112        tf->trapno == T_IRQ0+IRQ_TIMER)
110 113       yield();
    114 + #endif
111 115
112 116     // Check if the process has been killed since we yielded
113 117     if(myproc() && myproc()->killed && (tf->cs&3) == DPL_USER)
```

Param.h

param.h

```
@@ -11,4 +11,9 @@
11  11  #define LOGSIZE      (MAXOPBLOCKS*3)  // max data blocks in on-disk log
12  12  #define NBUF         (MAXOPBLOCKS*3)  // size of disk block cache
13  13  #define FSSIZE       1000  // size of file system in blocks
    14  + #define SJF_MINLEN      1  // shortest predicted job length (in ticks) the SJF scheduler hands out
    15  + #define SJF_MAXLEN    100  // longest predicted job length (in ticks)
    16  + #define SJF_SEED    12345  // starting seed of the predicted-length generator
    17  + #define SJF_LCG_A 1664525  // linear congruential generator multiplier
    18  + #define SJF_LCG_C 1013904223  // linear congruential generator increment
14  19
```

Part 4: Advanced scheduler, priority with round robin

make xv6.img SCHEDULER=[PRIORITYRR]: -DPRIORITYRR and quantum 5

```
ankrit@DESKTOP-IGT6P4O:~/xv6-public$ make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*.o *.d *.asm *.sym vectors.S bootblock entryother \
initcode initcode.out kernel xv6.img fs.img kernelmemfs \
xv6memfs.img mkfs .gdbinit \
_cat _echo _forktest _grep _init _kill _ln _ls _mkdir _rm _sh _stressfs _usertests _wc _zombie _hello _sleep _sort _tickstest _sjftest _timeit _priotest  .s*
ankrit@DESKTOP-IGT6P4O:~/xv6-public$
ankrit@DESKTOP-IGT6P4O:~/xv6-public$
ankrit@DESKTOP-IGT6P4O:~/xv6-public$ make xv6.img 'SCHEDULER=[PRIORITYRR]' 2>&1 | grep -oE "\-D[A-Z]+ -DRR_QUANTUM=[0-9]+ +-c -o [a-z]+\.o [a-z]+\.c" | grep"
-DPRIORITYRR -DRR_QUANTUM=5   -c -o proc.o proc.c
-DPRIORITYRR -DRR_QUANTUM=5   -c -o trap.o trap.c
```

PriorityRR output priotest

```
$
$ priotest
get_sched_priority(self)      = 1 (default level)
set_sched_priority(0)         = 0, now get = 0
set_sched_priority(3)         = -1 (expect -1, level unchanged: 0)
set_sched_priority(-1)        = -1 (expect -1)
get_sched_priority(9999)      = -1 (expect -1)
back at level                 = 1
starting 6 children, 30 ticks of CPU each...
finishing order (first to last):
  pid 7  priority 0  finished at tick 1392  slices 5
  pid 8  priority 0  finished at tick 1392  slices 5
  pid 9  priority 1  finished at tick 1452  slices 5
  pid 10  priority 1  finished at tick 1452  slices 5
  pid 11  priority 2  finished at tick 1512  slices 5
  pid 12  priority 2  finished at tick 1512  slices 5
after wait, get_sched_priority(first child) = -1 (expect -1)
$
```

Output of **timeit sressfs**

```
$
$ timeit stressfs
stressfs starting
write 0
write 1
write 2
write 3
write 4
read
read
read
read
read
timeit: stressfs took 38 ticks
$
```

Output of **timeit ls**

```
$
$ timeit ls
README          2 2 3405
os611.txt       2 3 149
nums.txt        2 4 14
cat             2 5 16228
echo            2 6 15120
forktest        2 7 9176
grep            2 8 22052
init            2 9 15728
kill            2 10 15196
ln              2 11 15060
ls              2 12 18248
mkdir           2 13 15192
rm              2 14 15172
sh              2 15 29184
stressfs        2 16 16020
usertests       2 17 63904
wc              2 18 16528
zombie          2 19 14760
hello           2 20 14936
sleep           2 21 15056
sort            2 22 18940
tickstest       2 23 16868
sjftest         2 24 17040
timeit          2 25 15544
priotest        2 26 19324
console         3 27 0
stressfs0       2 28 10240
stressfs1       2 29 10240
stressfs2       2 30 10240
stressfs3       2 31 10240
stressfs4       2 32 10240
timeit: ls took 39 ticks
$
```

```
$
$
$
$ cat README | wc
84 466 3405
$
$ forktest
fork test
fork test OK
$
$
```

Testing PRIORITYRR with quantum=10.

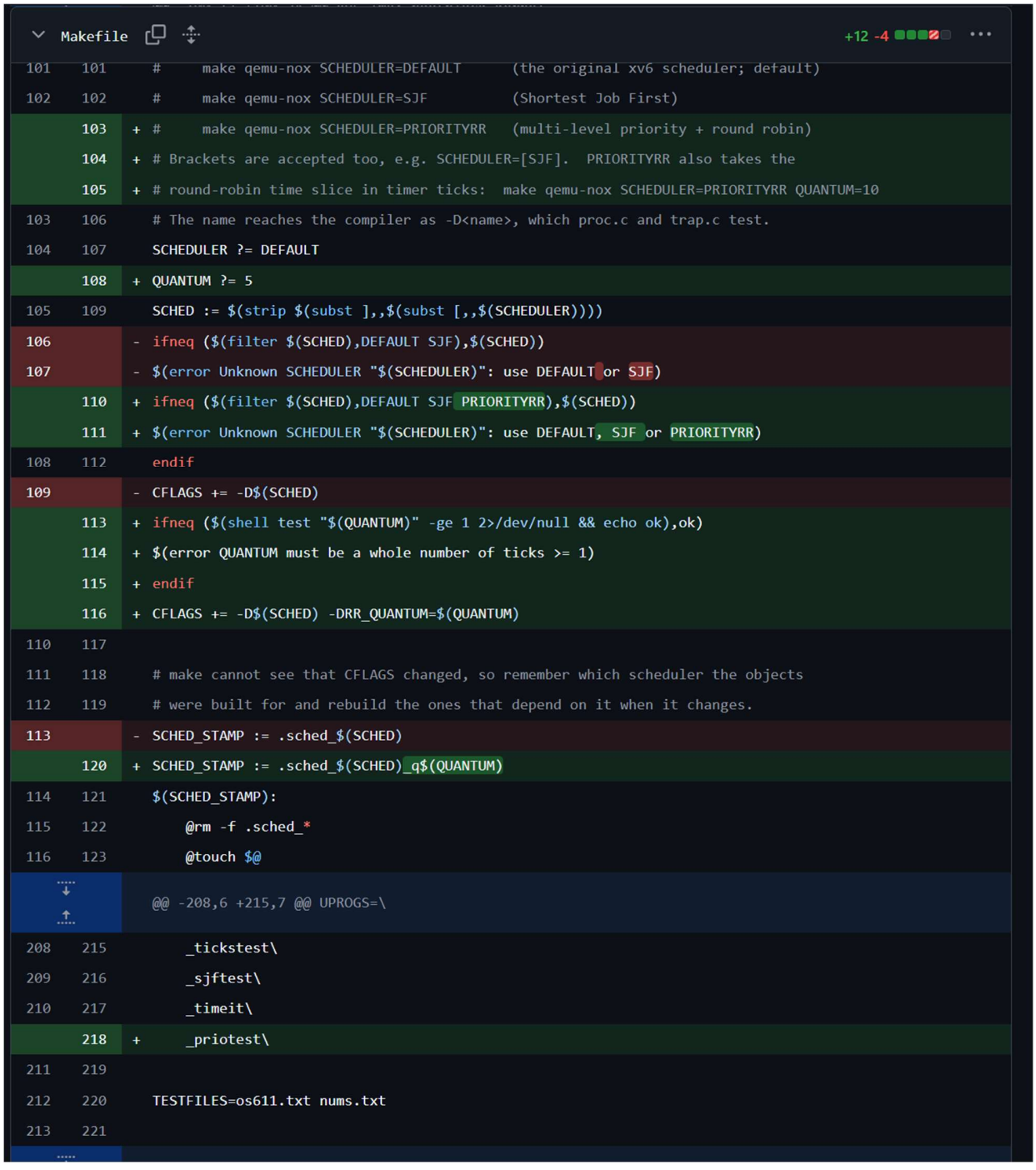
```
initcode initcode.out kernel xv6.img fs.img kernelmemfs \
xv6memfs.img mkfs .gdbinit \
ankrit@DESKTOP-IGT6P4O:~/xv6-public$ make qemu-nox SCHEDULER=PRIORITYRR QUANTUM=10 CPUS=1
gcc -Werror -Wall -o mkfs mkfs.c
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror -fno-omi
gcc -m32 -gdwarf-2 -Wa,-divide   -c -o usys.o usys.S
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror -fno-omi
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror -fno-omi
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror -fno-omi
ld -m    elf_i386 -N -e main -Ttext 0 -o _cat cat.o ulib.o usys.o printf.o umalloc.o
/usr/bin/x86_64-linux-gnu-ld.bfd: warning: _cat has a LOAD segment with RWX permissions
objdump -S _cat > cat.asm
objdump -t _cat | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > cat.sym
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror -fno-omi
ld -m    elf_i386 -N -e main -Ttext 0 -o _echo echo.o ulib.o usys.o printf.o umalloc.o
/usr/bin/x86_64-linux-gnu-ld.bfd: warning: _echo has a LOAD segment with RWX permissions
objdump -S _echo > echo.asm
objdump -t _echo | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > echo.sym
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror -fno-omi
# forktest has less library code linked in - needs to be small
# in order to be able to max out the proc table.
ld -m    elf_i386 -N -e main -Ttext 0 -o _forktest forktest.o ulib.o usys.o
/usr/bin/x86_64-linux-gnu-ld.bfd: warning: _forktest has a LOAD segment with RWX permissions
objdump -S _forktest > forktest.asm
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror -fno-omi
ld -m    elf_i386 -N -e main -Ttext 0 -o _grep grep.o ulib.o usys.o printf.o umalloc.o
```

Output of **priotest with quantum 10**

```
$
$ priotest
get_sched_priority(self)      = 1 (default level)
set_sched_priority(0)         = 0, now get = 0
set_sched_priority(3)         = -1 (expect -1, level unchanged: 0)
set_sched_priority(-1)        = -1 (expect -1)
get_sched_priority(9999)      = -1 (expect -1)
back at level                 = 1
starting 6 children, 30 ticks of CPU each...
finishing order (first to last):
  pid 6  priority 0  finished at tick 2616  slices 2
  pid 7  priority 0  finished at tick 2616  slices 2
  pid 8  priority 1  finished at tick 2676  slices 3
  pid 9  priority 1  finished at tick 2676  slices 2
  pid 10  priority 2  finished at tick 2736  slices 3
  pid 11  priority 2  finished at tick 2736  slices 2
after wait, get_sched_priority(first child) = -1 (expect -1)
$
```

Code Changes:

Makefile



Makefile +12 -4

```
101 101   #     make qemu-nox SCHEDULER=DEFAULT      (the original xv6 scheduler; default)
102 102   #     make qemu-nox SCHEDULER=SJF          (Shortest Job First)
    103 + #     make qemu-nox SCHEDULER=PRIORITYRR   (multi-level priority + round robin)
    104 + # Brackets are accepted too, e.g. SCHEDULER=[SJF].  PRIORITYRR also takes the
    105 + # round-robin time slice in timer ticks:  make qemu-nox SCHEDULER=PRIORITYRR QUANTUM=10
103 106   # The name reaches the compiler as -D<name>, which proc.c and trap.c test.
104 107   SCHEDULER ?= DEFAULT
    108 + QUANTUM ?= 5
105 109   SCHED := $(strip $(subst ],,$(subst [,,$(SCHEDULER))))
106     - ifneq ($(filter $(SCHED),DEFAULT SJF),$(SCHED))
107     - $(error Unknown SCHEDULER "$(SCHEDULER)": use DEFAULT or SJF)
    110 + ifneq ($(filter $(SCHED),DEFAULT SJF PRIORITYRR),$(SCHED))
    111 + $(error Unknown SCHEDULER "$(SCHEDULER)": use DEFAULT, SJF or PRIORITYRR)
108 112   endif
109     - CFLAGS += -D$(SCHED)
    113 + ifneq ($(shell test "$(QUANTUM)" -ge 1 2>/dev/null && echo ok),ok)
    114 + $(error QUANTUM must be a whole number of ticks >= 1)
    115 + endif
    116 + CFLAGS += -D$(SCHED) -DRR_QUANTUM=$(QUANTUM)
110 117
111 118   # make cannot see that CFLAGS changed, so remember which scheduler the objects
112 119   # were built for and rebuild the ones that depend on it when it changes.
113     - SCHED_STAMP := .sched_$(SCHED)
    120 + SCHED_STAMP := .sched_$(SCHED)_q$(QUANTUM)
114 121   $(SCHED_STAMP):
115 122       @rm -f .sched_*
116 123       @touch $@
        @@ -208,6 +215,7 @@ UPROGS=\
208 215       _tickstest\
209 216       _sjftest\
210 217       _timeit\
    218 +     _priotest\
211 219
212 220   TESTFILES=os611.txt nums.txt
213 221
```

Priotest.c

priotest.c +134

```
+ // priotest: exercises set_sched_priority() / get_sched_priority() and shows
+ // the PRIORITYRR scheduler at work.
+ //   usage: priotest [work_ticks]        (default 30)
+ // Six children, two at each priority level (0 = highest), each burn
+ // work_ticks of CPU.  Expected under SCHEDULER=PRIORITYRR:
+ //   * the level-0 pair finishes first, then level 1, then level 2;
+ //   * the two children inside a level finish close together because they
+ //     take turns (round robin); "slices" counts the times a child was taken
+ //     off the CPU for a while, roughly work_ticks / quantum.
+ // Under the default scheduler priorities are ignored and everything
+ // finishes together.
+ #include "types.h"
+ #include "stat.h"
+ #include "user.h"
+
+ #define LEVELS        3     // priority levels exercised (matches PRIORITY_LEVELS)
+ #define PER_LEVEL     2     // children per level
+ #define DEFAULT_WORK  30
+
+ struct rec {
+   int pid;
+   int prio;
+   int tick;
+   int slices;
+ };
+
+ static void
+ child(int prio, int work, int readyfd, int gofd, int wfd)
+ {
+   struct rec r;
+   char c = 'r';
+   int last, now, slices = 0;
+
+   if(set_sched_priority(prio) < 0){
+     printf(1, "child: set_sched_priority(%d) failed\n", prio);
+     exit();
+   }
```

priotest.c +134

```
+   // that all the children become runnable at the same moment.
+   write(readyfd, &c, 1);
+   read(gofd, &c, 1);
+   // Burn CPU until we have been RUNNING for `work` ticks.  A jump of more
+   // than one tick between two looks at the clock means we were descheduled.
+   last = uptime();
+   while(ticks_running(getpid()) < work){
+     now = uptime();
+     if(now - last > 1)
+       slices++;
+     last = now;
+   }
+   r.pid = getpid();
+   r.prio = get_sched_priority(getpid());
+   r.tick = uptime();
+   r.slices = slices;
+   write(wfd, &r, sizeof(r));
+   exit();
+ }
+
+ int
+ main(int argc, char *argv[])
+ {
+   int work = DEFAULT_WORK;
+   int p[2], ready[2], go[2], me = getpid();
+   int i, level, n = LEVELS * PER_LEVEL;
+   int rv, gv;
+   char c = 'g';
+   struct rec r;
+
+   if(argc > 1)
+     work = atoi(argv[1]);
+   if(work < 1){
+     printf(1, "usage: priotest [work_ticks]\n");
+     exit();
+   }
```

priotest.c +134

```
74  +   }
75  +
76  +   // ---- 1. the system calls themselves ----
77  +   // (each call is made on its own line: printf's arguments are evaluated
78  +   // in no particular order, which would scramble the before/after values)
79  +   gv = get_sched_priority(me);
80  +   printf(1, "get_sched_priority(self)      = %d (default level)\n", gv);
81  +   rv = set_sched_priority(0);
82  +   gv = get_sched_priority(me);
83  +   printf(1, "set_sched_priority(0)         = %d, now get = %d\n", rv, gv);
84  +   rv = set_sched_priority(LEVELS);
85  +   gv = get_sched_priority(me);
86  +   printf(1, "set_sched_priority(%d)         = %d (expect -1, level unchanged: %d)\n", LEVELS, rv,
        gv);
87  +   rv = set_sched_priority(-1);
88  +   printf(1, "set_sched_priority(-1)        = %d (expect -1)\n", rv);
89  +   rv = get_sched_priority(9999);
90  +   printf(1, "get_sched_priority(9999)      = %d (expect -1)\n", rv);
91  +   set_sched_priority(1);
92  +   gv = get_sched_priority(me);
93  +   printf(1, "back at level                 = %d\n", gv);
94  +
95  +   // ---- 2. scheduling order ----
96  +   if(pipe(p) < 0 || pipe(ready) < 0 || pipe(go) < 0){
97  +     printf(1, "pipe failed\n");
98  +     exit();
99  +   }
100 +   printf(1, "starting %d children, %d ticks of CPU each...\n", n, work);
101 +   for(level = 0; level < LEVELS; level++){
102 +     for(i = 0; i < PER_LEVEL; i++){
103 +       int pid = fork();
104 +       if(pid < 0){
105 +         printf(1, "fork failed\n");
106 +         exit();
107 +       }
108 +       if(pid == 0){
```

priotest.c +134

```
106 +         exit();
107 +       }
108 +       if(pid == 0){
109 +         close(p[0]);
110 +         close(ready[0]);
111 +         close(go[1]);
112 +         child(level, work, ready[1], go[0], p[1]);
113 +       }
114 +     }
115 +   }
116 +   close(p[1]);
117 +   close(ready[1]);
118 +   close(go[0]);
119 +   for(i = 0; i < n; i++)          // wait until every child is at its level
120 +     read(ready[0], &c, 1);
121 +   for(i = 0; i < n; i++)          // ...then release them all together
122 +     write(go[1], &c, 1);
123 +   for(i = 0; i < n; i++)
124 +     wait();
125 +
126 +   printf(1, "finishing order (first to last):\n");
127 +   while(read(p[0], &r, sizeof(r)) == sizeof(r))
128 +     printf(1, "  pid %d  priority %d  finished at tick %d  slices %d\n",
129 +            r.pid, r.prio, r.tick, r.slices);
130 +   close(p[0]);
131 +   rv = get_sched_priority(me + 1);
132 +   printf(1, "after wait, get_sched_priority(first child) = %d (expect -1)\n", rv);
133 +   exit();
134 + }
```

Proc.c changes

proc.c +96 -2

```
@@ -20,6 +20,13 @@ extern void trapret(void);

static void wakeup1(void *chan);

+ // PRIORITYRR: the table slot that ran last at each priority level, so the
+ // next pass at that level starts just after it (round robin).  Protected by
+ // ptable.lock.
+ #ifdef PRIORITYRR
+ static int rr_last[PRIORITY_LEVELS];
+ #endif
+
// Predicted job lengths for the SJF scheduler come from a simple linear
// congruential generator, so they vary from process to process but are
// repeatable from boot to boot.  Callers hold ptable.lock.

@@ -102,6 +109,8 @@ allocproc(void)
  p->pid = nextpid++;
  p->run_ticks = 0;
  p->job_len = next_job_len();
+  p->priority = PRIORITY_DEFAULT;
+  p->quantum_used = 0;

  release(&ptable.lock);

@@ -212,6 +221,7 @@ fork(void)
  }
  np->sz = curproc->sz;
  np->parent = curproc;
+  np->priority = curproc->priority;   // a child starts at its parent's priority
  *np->tf = *curproc->tf;

  // Clear %eax so that fork returns 0 in the child.

@@ -337,8 +347,10 @@ void
```

```
                struct proc *p;
-  #ifdef SJF
+  #if defined(SJF)
     struct proc *best;
+  #elif defined(PRIORITYRR)
+    int level, i, slot;
   #endif
     struct cpu *c = mycpu();
     c->proc = 0;
   @@ -348,7 +360,7 @@ scheduler(void)
       sti();

       acquire(&ptable.lock);
-  #ifdef SJF
+  #if defined(SJF)
       // Shortest Job First: of all RUNNABLE processes, run the one with the
       // smallest predicted job length (ties go to the lowest table slot).
       // trap.c does not preempt on timer ticks in this mode, so the chosen
   @@ -369,6 +381,35 @@ scheduler(void)
         swtch(&(c->scheduler), p->context);
         switchkvm();

+       c->proc = 0;
+     }
+  #elif defined(PRIORITYRR)
+     // Multi-level priority with round robin inside each level.  Take the
+     // highest non-empty level (0 is highest) and, within it, the next
+     // RUNNABLE process after the one that ran last at that level.
+     // trap.c preempts on a timer tick when the quantum is used up or a
+     // higher-priority process has become RUNNABLE.
+     p = 0;
+     for(level = 0; level < PRIORITY_LEVELS && p == 0; level++){
+       for(i = 1; i <= NPROC; i++){
+         slot = (rr_last[level] + i) % NPROC;
+         if(ptable.proc[slot].state == RUNNABLE &&
+            ptable.proc[slot].priority == level){
```

```
371  383
     384  +       c->proc = 0;
     385  +     }
     386  + #elif defined(PRIORITYRR)
     387  +     // Multi-level priority with round robin inside each level.  Take the
     388  +     // highest non-empty level (0 is highest) and, within it, the next
     389  +     // RUNNABLE process after the one that ran last at that level.
     390  +     // trap.c preempts on a timer tick when the quantum is used up or a
     391  +     // higher-priority process has become RUNNABLE.
     392  +     p = 0;
     393  +     for(level = 0; level < PRIORITY_LEVELS && p == 0; level++){
     394  +       for(i = 1; i <= NPROC; i++){
     395  +         slot = (rr_last[level] + i) % NPROC;
     396  +         if(ptable.proc[slot].state == RUNNABLE &&
     397  +            ptable.proc[slot].priority == level){
     398  +           p = &ptable.proc[slot];
     399  +           rr_last[level] = slot;
     400  +           break;
     401  +         }
     402  +       }
     403  +     }
     404  +     if(p != 0){
     405  +       p->quantum_used = 0;
     406  +       c->proc = p;
     407  +       switchuvm(p);
     408  +       p->state = RUNNING;
     409  +
     410  +       swtch(&(c->scheduler), p->context);
     411  +       switchkvm();
     412  +
372  413         c->proc = 0;
373  414       }
```

proc.c +96 -2

```
412 +
372 413         c->proc = 0;
373 414       }
374 415   #else
        @@ -577,6 +618,59 @@ procjoblen(int pid)
577 618     return len;
578 619   }
579 620
621 + // Called from the timer interrupt (PRIORITYRR).  Charges this tick to the
622 + // running process and says whether it should give up the CPU now: its time
623 + // quantum is used up, or a process with a higher priority is waiting.
624 + int
625 + sched_should_yield(void)
626 + {
627 +   struct proc *p, *cur = myproc();
628 +   int yield_now = 0;
629 +
630 +   acquire(&ptable.lock);
631 +   cur->quantum_used++;
632 +   if(cur->quantum_used >= RR_QUANTUM)
633 +     yield_now = 1;
634 +   for(p = ptable.proc; p < &ptable.proc[NPROC] && !yield_now; p++){
635 +     if(p->state == RUNNABLE && p->priority < cur->priority)
636 +       yield_now = 1;
637 +   }
638 +   release(&ptable.lock);
639 +   return yield_now;
640 + }
641 +
```

proc.c +96 -2

```
642 + // Set the priority of the calling process.  Returns 0, or -1 if the level is
643 + // outside 0 .. PRIORITY_LEVELS-1.
644 + int
645 + setprio(int prio)
646 + {
647 +   if(prio < 0 || prio >= PRIORITY_LEVELS)
648 +     return -1;
649 +   acquire(&ptable.lock);
650 +   myproc()->priority = prio;
651 +   release(&ptable.lock);
652 +   return 0;
653 + }
654 +
655 + // Return the priority of process pid, or -1 if there is no such process or it
656 + // is not in the scheduling queue (RUNNABLE or RUNNING).
657 + int
658 + getprio(int pid)
659 + {
660 +   struct proc *p;
661 +   int prio = -1;
662 +
663 +   acquire(&ptable.lock);
664 +   for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
665 +     if(p->pid == pid && (p->state == RUNNABLE || p->state == RUNNING)){
666 +       prio = p->priority;
667 +       break;
668 +     }
669 +   }
670 +   release(&ptable.lock);
671 +   return prio;
672 + }
673 +
580 674   // Print a process listing to console.  For debugging.
581 675   // Runs when user types ^P on console.
582 676   // No lock to avoid wedging a stuck machine further.
```

Sysproc.c changes

sysproc.c +20

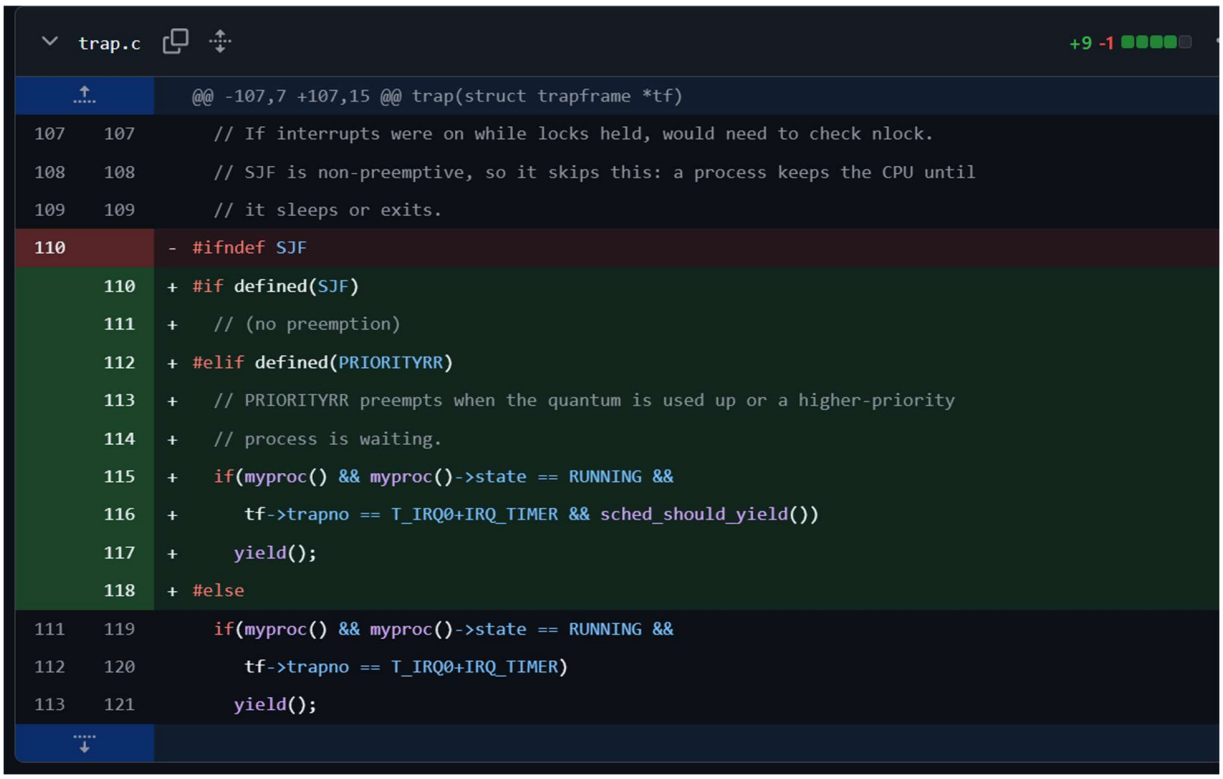
```
@@ -118,3 +118,23 @@ sys_sjf_job_length(void)
118 118     return -1;
119 119   return procjoblen(pid);
120 120   }
    121 +
    122 + int
    123 + sys_set_sched_priority(void)
    124 + {
    125 +   int prio;
    126 +
    127 +   if(argint(0, &prio) < 0)
    128 +     return -1;
    129 +   return setprio(prio);
    130 + }
    131 +
    132 + int
    133 + sys_get_sched_priority(void)
    134 + {
    135 +   int pid;
    136 +
    137 +   if(argint(0, &pid) < 0)
    138 +     return -1;
    139 +   return getprio(pid);
    140 + }
```

Param.h

param.h

```
@@ -10,10 +10,15 @@
10 10  #define MAXOPBLOCKS  10  // max # of blocks any FS op writes
11 11  #define LOGSIZE      (MAXOPBLOCKS*3)  // max data blocks in on-disk log
12 12  #define NBUF         (MAXOPBLOCKS*3)  // size of disk block cache
13    - #define FSSIZE       1000  // size of file system in blocks
    13 + #define FSSIZE       2000  // size of file system in blocks
14 14  #define SJF_MINLEN      1  // shortest predicted job length (in ticks) the SJF scheduler hands out
15 15  #define SJF_MAXLEN    100  // longest predicted job length (in ticks)
16 16  #define SJF_SEED    12345  // starting seed of the predicted-length generator
17 17  #define SJF_LCG_A 1664525  // linear congruential generator multiplier
18 18  #define SJF_LCG_C 1013904223  // linear congruential generator increment
    19 + #define PRIORITY_LEVELS  3  // priority levels for PRIORITYRR: 0 (highest) .. PRIORITY_LEVELS-1 (lowest)
    20 + #define PRIORITY_DEFAULT 1  // level every new process starts at (children inherit their parent's)
    21 + #ifndef RR_QUANTUM
    22 + #define RR_QUANTUM       5  // round-robin time quantum in timer ticks; override with make QUANTUM=n
    23 + #endif
19 24
```

Trap.c



trap.c +9 -1

```
@@ -107,7 +107,15 @@ trap(struct trapframe *tf)
107 107     // If interrupts were on while locks held, would need to check nlock.
108 108     // SJF is non-preemptive, so it skips this: a process keeps the CPU until
109 109     // it sleeps or exits.
110       - #ifndef SJF
      110 + #if defined(SJF)
      111 +   // (no preemption)
      112 + #elif defined(PRIORITYRR)
      113 +   // PRIORITYRR preempts when the quantum is used up or a higher-priority
      114 +   // process is waiting.
      115 +   if(myproc() && myproc()->state == RUNNING &&
      116 +      tf->trapno == T_IRQ0+IRQ_TIMER && sched_should_yield())
      117 +     yield();
      118 + #else
111 119     if(myproc() && myproc()->state == RUNNING &&
112 120        tf->trapno == T_IRQ0+IRQ_TIMER)
113 121       yield();
```

Git log for changes.

```
nothing to commit, working tree clean
Everything up-to-date
63d5089 (HEAD -> assignment-2, origin/assignment-2) Part 4: PRIORITYRR schu0
87e447e Part 3: SJF scheduler, sjf_job_length syscall, sjftest and timeit
7832e35 Part 2: add ticks_running(pid) syscall and tickstest
49b9415 Part 1: extend grep with -i, -n, -v flags
4570f8d Part 1: add sort (-r, -n, -u, stdin) and bake test files into fs.img
1a02f37 (origin/main, origin/assignment-1, origin/HEAD, main, assignment-1Ad
```